

Metody Obliczeniowe w Nauce i Technice

Układy równań liniowych – metody bezpośrednie

Katarzyna Piskorz | 13 maja 2026 | Informatyka

1. Wstęp teoretyczny i omówienie przypadku

W ramach laboratorium przeprowadzono analizę numerycznego rozwiązywania układów równań liniowych postaci:

$$Ax = b$$

gdzie:

- A – kwadratowa macierz współczynników układu o wymiarach $n \times n$,
- x – wektor kolumnowy poszukiwanych niewiadomych o długości n ,
- b – wektor wyrazów wolnych o długości n .

Celem pracy było zbadanie wpływu **uwarunkowania macierzy** oraz **precyzji obliczeń** (float32 vs float64) na dokładność rozwiązań, a także porównanie klasycznej **eliminacji Gaussa** z **algorytmem Thomasa**. Badaniom poddano trzy rodzaje macierzy, zdefiniowane poniżej (w każdym przypadku i oraz j oznaczają odpowiednio indeks wiersza i indeks kolumny macierzy, przyjmujące wartości od 1 do n):

Macierz I (zbliżona do macierzy Hilberta):

$$A_1 = \begin{cases} a_{1j} = 1 \\ a_{ij} = \frac{1}{i+j-1} \end{cases} \quad \text{dla } i \neq 1$$

Gdzie a_{ij} stanowi pojedynczy element macierzy A_1 . Wzór ten tworzy macierz o szybko malejących ułamkach w miarę przesuwania się w dół i w prawo po siatce elementów.

Macierz II:

$$A_2 = \begin{cases} a_{ij} = \frac{2i}{j} & \text{dla } j \geq i \\ a_{ij} = a_{ji} & \text{dla } j < i \end{cases}$$

Gdzie a_{ij} to wyraz macierzy A_2 . Drugi warunek gwarantuje jej symetryczność względem głównej przekątnej.

Macierz III (trójdzielna, parametry indywidualne $k = 5, m = 4$):

$$A_3 = \begin{cases} a_{i,i} = -4i - 5 \\ a_{i,i-1} = \frac{i}{4} & \text{dla } i > 1 \\ a_{i,i+1} = \frac{i}{4} & \text{dla } i < n \\ a_{i,j} = 0 & \text{dla } j < i - 1 \text{ oraz } j > i + 1 \end{cases}$$

W macierzy trójdzielnej A_3 niezerowe elementy występują wyłącznie na **głównej przekątnej** ($a_{i,i}$), bezpośrednio **pod nią** ($a_{i,i-1}$) oraz bezpośrednio **nad nią** ($a_{i,i+1}$). Pozostałe elementy ($a_{i,j}$ dla $|i - j| > 1$) są równe zero.

2. Metodyka badawcza

Do weryfikacji poprawności zaimplementowanych algorytmów zastosowano tzw. **metodę odwróconą**. Polega ona na ustaleniu prawdziwego wyniku z góry. Wektor rozwiązania zadanego x_{zad} został wygenerowany jako ciąg n elementów losowanych z rozkładu jednostajnego ze zbioru $\{1, -1\}$. Aby zapewnić **determinizm eksperymentów numerycznych** i powtarzalność wyników (co jest kluczowe przy analizie załamań precyzji zmiennoprzecinkowej), zablokowano ziarno generatora liczb pseudolosowych na stałej wartości (`seed = 42`). Dzięki temu przy każdym uruchomieniu i dla każdego formatu danych (`float32` / `float64`) wektor x_{zad} pozostawał dokładnie taki sam, umożliwiając obiektywne porównanie błędów poszczególnych algorytmów.

Następnie na podstawie zadanego wektora wyliczono sztuczny wektor wyrazów wolnych b :

$$b = A \cdot x_{\text{zad}}$$

Z tak spreparowanym wektorem b , program uruchamiał algorytm rozwiązujący układ $Ax = b$, generując wyliczony wektor x_{obl} . Jako miarę dokładności algorytmu przyjęto **bezwzględny błąd maksymalny** (metrykę zbieżną z normą nieskończoność), definiowany jako:

$$\text{Błąd}_{\text{max}} = \|x_{\text{zad}} - x_{\text{obl}}\|_{\infty} = \max_{1 \leq i \leq n} |x_{\text{zad},i} - x_{\text{obl},i}|$$

gdzie $x_{\text{zad},i}$ oraz $x_{\text{obl},i}$ to poszczególne (i -te) składowe odpowiednich wektorów. Wykorzystanie maksimum różnic wyłapuje **najgorsze, punktowe załamanie dokładności** w całym układzie.

Dodatkowo do analizy podatności numerycznej układu wykorzystano **wskaźnik uwarunkowania macierzy** (ang. **condition number**):

$$\text{cond}(A) = \|A\|_1 \cdot \|A^{-1}\|_1$$

Gdzie $\|A\|_1$ to 1-norma zdefiniowana jako maksymalna suma wartości bezwzględnych z elementów w kolumnie, a A^{-1} to macierz odwrotna do macierzy A .

Z matematycznego punktu widzenia, związek między uwarunkowaniem a propagacją błędów względnego wyraża **fundamentalna nierówność**:

$$\frac{\|\Delta x\|}{\|x\|} \leq \text{cond}(A) \cdot \frac{\|\Delta A\|}{\|A\|}$$

Równanie to pokazuje wprost, dlaczego duży wskaźnik $\text{cond}(A)$ potrafi drastycznie zaburzyć błąd wyliczonego wektora x — nawet w sytuacji, gdy błąd reprezentacji maszynowej ΔA jest rzędu zaledwie 10^{-16} (jak ma to miejsce w arytmetyce podwójnej precyzji).

2.1. Wykorzystane narzędzia i biblioteki

Programy badawcze napisano w języku Python. Do implementacji algorytmów oraz monitorowania ich wydajności użyto następujących modułów wbudowanych i zewnętrznych:

- **numpy.linalg.cond** – funkcja z biblioteki NumPy posłużyła do wyliczenia wskaźnika uwarunkowania badanych macierzy.
- **time.perf_counter** – funkcja służąca do precyzyjnego pomiaru czasu wykonania samych bloków operacyjnych algorytmów (bez pomiaru czasu generacji danych wejściowych). Posiada ona najwyższą dostępną w Pythonie rozdzielczość zegara (rzędu nanosekund).
- **tracemalloc** – moduł biblioteki standardowej pozwalający na profilowanie pamięci operacyjnej. Funkcja `get_traced_memory()` została użyta do zmierzenia szczytowego zapotrzebowania na pamięć RAM (w kilobajtach) podczas fizycznego wykonywania się algorytmów.

- **Tablice `numpy.ndarray`** – podstawowa struktura danych, w której w pamięci operacyjnej fizycznie przechowywano macierze poddawane obliczeniom.

3. Analiza i wnioski: Złe uwarunkowanie (Zadanie 1)

W pierwszym ćwiczeniu badano zachowanie metody Gaussa dla macierzy A_1 , której struktura czyni ją kompletnie źle uwarunkowaną.

Aby lepiej zobrazować naturę badanych problemów numerycznych, poniżej przedstawiono fizyczny zrzut pamięci (dla ułamków zaokrąglonych do 4 miejsc po przecinku) wygenerowanej macierzy A_1 o rozmiarze $n = 5$:

$$A_1 = \begin{pmatrix} 1.0 & 1.0 & 1.0 & 1.0 & 1.0 \\ 0.5 & 0.3333 & 0.25 & 0.2 & 0.1667 \\ 0.3333 & 0.25 & 0.2 & 0.1667 & 0.1429 \\ 0.25 & 0.2 & 0.1667 & 0.1429 & 0.125 \\ 0.2 & 0.1667 & 0.1429 & 0.125 & 0.1111 \end{pmatrix}$$

W poniższej tabeli kolorem czerwonym wyróżniono wartości krytyczne: błędy maksymalne, które przekroczyły wartość 1.0 (co oznacza całkowitą utratę poprawności algorytmu, jako że szukane wartości to 1 lub -1) oraz ekstremalne wskaźniki uwarunkowania dla największej próby.

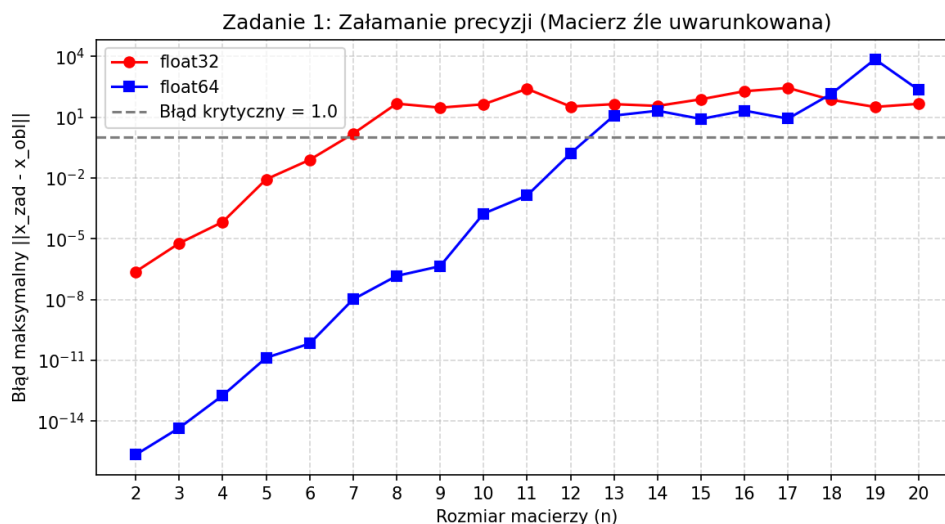
Tabela 1: Szczegółowe wyniki rozwiązania układu $A_1x = b$ metodą Gaussa

n	Błąd max (f32)	Cond (f32)	Czas [s] (f32)	Błąd max (f64)	Cond (f64)	Czas [s] (f64)
2	2.38e-07	1.80e+01	1.72e-04	2.22e-16	1.80e+01	1.20e-04
3	5.96e-06	6.60e+02	2.16e-04	4.66e-15	6.60e+02	2.03e-04
4	6.75e-05	2.00e+04	2.93e-04	1.86e-13	2.00e+04	2.46e-04
5	8.55e-03	7.99e+05	3.75e-04	1.35e-11	7.98e+05	3.08e-04
6	7.80e-02	2.81e+07	4.34e-04	6.74e-11	2.73e+07	4.09e-04
7	1.51e+00	1.64e+09	5.39e-04	1.03e-08	8.58e+08	5.11e-04
8	4.62e+01	3.87e+09	6.91e-04	1.46e-07	2.99e+10	6.32e-04
9	2.93e+01	1.24e+10	8.38e-04	4.42e-07	1.04e+12	7.72e-04
10	4.29e+01	4.99e+09	9.96e-04	1.74e-04	3.38e+13	9.96e-04
11	2.49e+02	3.36e+09	1.20e-03	1.36e-03	1.11e+15	1.08e-03
12	3.32e+01	3.58e+09	1.28e-03	1.63e-01	3.87e+16	1.31e-03
13	4.36e+01	5.49e+09	1.50e-03	1.19e+01	3.70e+17	1.55e-03
14	3.57e+01	4.67e+09	1.74e-03	2.05e+01	4.01e+17	1.72e-03
15	7.58e+01	9.28e+09	1.91e-03	8.11e+00	3.52e+17	2.00e-03
16	1.89e+02	1.52e+10	2.17e-03	2.08e+01	1.02e+18	2.25e-03
17	2.74e+02	2.64e+10	2.39e-03	8.36e+00	1.98e+18	2.52e-03
18	7.08e+01	1.87e+10	2.61e-03	1.42e+02	3.05e+18	2.72e-03
19	3.18e+01	2.05e+10	5.46e-03	7.13e+03	3.29e+18	2.99e-03
20	4.52e+01	3.20e+10	3.29e-03	2.23e+02	4.24e+18	3.23e-03

Wnioski:

- **Błąd biblioteczny a fałszywy wynik cond:** W Tabeli 1 widać, że dla $n = 20$ pomiar float32 wynosi ok. 3.20×10^{10} , podczas gdy dla float64 to aż **4.24×10^{18}** . Matematycznie rzecz biorąc, wskaźnik uwarunkowania zależy wyłącznie od macierzy, więc powinien

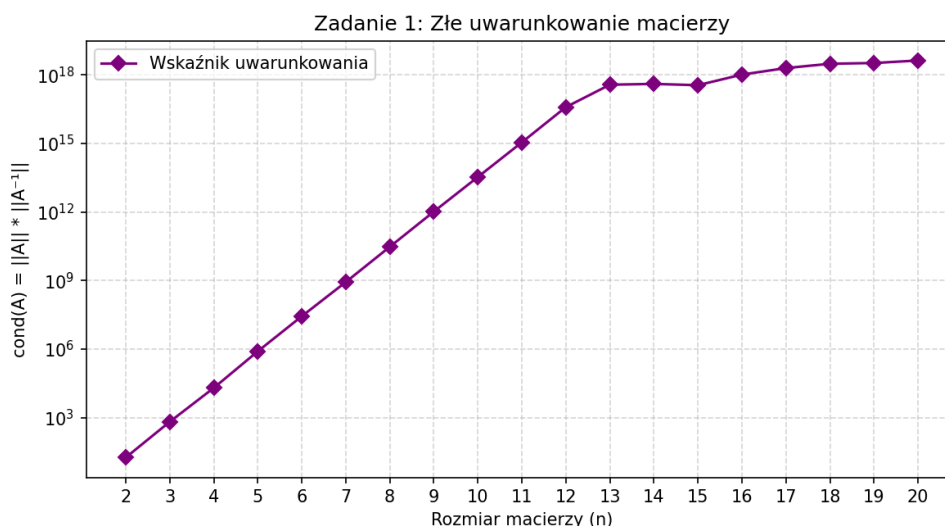
być identyczny. Różnica ta dowodzi, że dla tak dużych wielkości n macierz staje się tak **ekstremalnie źle uwarunkowana**, że funkcja `numpy.linalg.cond`, która wewnątrz algorytmu wylicza macierz odwrotną, nie ma w `float32` wystarczającej liczby bitów, aby przechować dokładność operacji. W efekcie wylicza fałszywy, mocno zaniżony wynik. Tylko pojemność zmiennej `float64` pozwoliła tutaj dotrzeć w okolice rzeczywistej, analitycznej wartości rzędu 10^{18} .



Rysunek 1: Załamanie precyzji obliczeń (błąd maks.)

Wnioski:

- **Załamanie precyzji obliczeń:** Na wykresie błędów (Rys. 1) oraz z rozkładu nowej tabeli idealnie widać moment załamania. Dla pojedynczej precyzji (`float32` - krzywa czerwona) błąd przekracza granicę sensu (szara linia > 1.0) **dokładnie przy $n = 7$** . W precyzji podwójnej (`float64` - krzywa niebieska), błąd rośnie wolniej, dzięki czemu algorytm poddaje się dopiero przy $n = 13$, gdzie błąd natychmiastowo wybija z ułamków do wartości 11.9.



Rysunek 2: (dla float64) Wykładniczy wzrost uwarunkowania - $O(e^n)$

Wnioski:

- **Wpływ wektora x_{zad} :** Zastosowanie sprzężonych ze sobą wartości 1 i -1 dodatkowo uwydatnia problem szumu. Wyliczenie wyrazu wolnego dla takich wartości wymusza naprze-

mienne odejmowanie i dodawanie bliskich sobie ułamków o ogromnych wykładnikach w macierzy, co prowadzi do zjawiska **katastrofalnego kasowania się cyfr znaczących**.

4. Analiza i wnioski: Dobrze uwarunkowanie (Zadanie 2)

Macierz A_2 zaprojektowano w taki sposób, aby jej struktura gwarantowała dobrą stabilność numeryczną.

Zadanie 2 (Fizyczny wycinek macierzy dobrze uwarunkowanej dla $n = 5$):

$$A_2 = \begin{pmatrix} 2.0 & 1.0 & 0.6667 & 0.5 & 0.4 \\ 1.0 & 2.0 & 1.3333 & 1.0 & 0.8 \\ 0.6667 & 1.3333 & 2.0 & 1.5 & 1.2 \\ 0.5 & 1.0 & 1.5 & 2.0 & 1.6 \\ 0.4 & 0.8 & 1.2 & 1.6 & 2.0 \end{pmatrix}$$

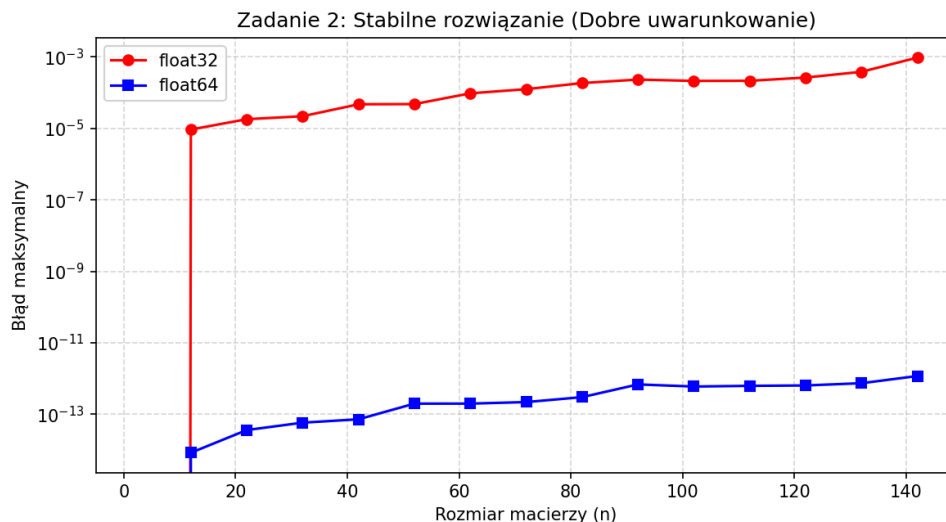
Tabela 2: Przekrój rozwiązania układu $A_2x = b$ metodą Gaussa z równym skokiem co 10

n	Błąd max (f32)	Cond (f32)	Czas [s] (f32)	Błąd max (f64)	Cond (f64)	Czas [s] (f64)
2	0.00e+00	3.00e+00	1.14e-04	0.00e+00	3.00e+00	1.27e-04
10	5.01e-06	1.15e+02	9.15e-04	6.66e-15	1.15e+02	9.68e-04
20	1.31e-05	4.73e+02	3.23e-03	4.05e-14	4.73e+02	3.19e-03
30	2.26e-05	1.07e+03	7.00e-03	6.39e-14	1.07e+03	7.07e-03
40	3.57e-05	1.92e+03	1.24e-02	8.02e-14	1.92e+03	1.23e-02
50	4.57e-05	3.00e+03	1.89e-02	1.94e-13	3.00e+03	1.88e-02
60	9.39e-05	4.33e+03	2.74e-02	1.71e-13	4.33e+03	2.72e-02
70	1.24e-04	5.90e+03	3.64e-02	1.86e-13	5.90e+03	3.70e-02
80	1.65e-04	7.71e+03	5.39e-02	2.89e-13	7.71e+03	4.82e-02
90	1.84e-04	9.77e+03	6.06e-02	6.76e-13	9.77e+03	6.08e-02
100	1.96e-04	1.21e+04	7.29e-02	6.35e-13	1.21e+04	7.53e-02
110	2.23e-04	1.46e+04	9.03e-02	5.30e-13	1.46e+04	8.99e-02
120	2.65e-04	1.74e+04	1.08e-01	6.50e-13	1.74e+04	1.08e-01
130	3.09e-04	2.04e+04	1.33e-01	7.50e-13	2.04e+04	1.25e-01
140	7.52e-04	2.37e+04	1.46e-01	9.80e-13	2.37e+04	1.46e-01
150	6.83e-04	2.72e+04	1.65e-01	9.01e-13	2.72e+04	1.68e-01

Wnioski:

- **Dlaczego uwarunkowania nagle stały się takie same (Upcasting pod maską):** O ile w zadaniu 1 wartości $\text{cond}(A)$ drastycznie się „rozjeżdżały”, o tyle w zadaniu 2 dla każdej precyzji w kolumnach są idealnie równe (np. 2.72×10^4 dla $n = 150$). Wynika to z matematycznej stabilności układu, ale też z tego, **jak działa funkcja biblioteczna `numpy.linalg.cond`**. Ponieważ wewnętrznie wykorzystuje ona skomplikowany algorytm dekompozycji SVD, to aby uniknąć pomyłki, automatycznie rzutuje mniejszą macierz `float32` na typ `float64` (tzw. upcasting), dodając zera do brakujących bitów ułamka. Dlatego wskaźnik uwarunkowania pokazywany dla `float32` w drugiej tabeli jest tak naprawdę rzetelnie policzonym, niezakłóconym wskaźnikiem z precyzji 64-bitowej.
- **Wysoka stabilność wyników:** Dzięki dobremu uwarunkowaniu eliminacja Gaussa nie potęguje błędów zaokrągleń. Błąd wyliczenia wektora w `float64` utrzymuje się na fenome-

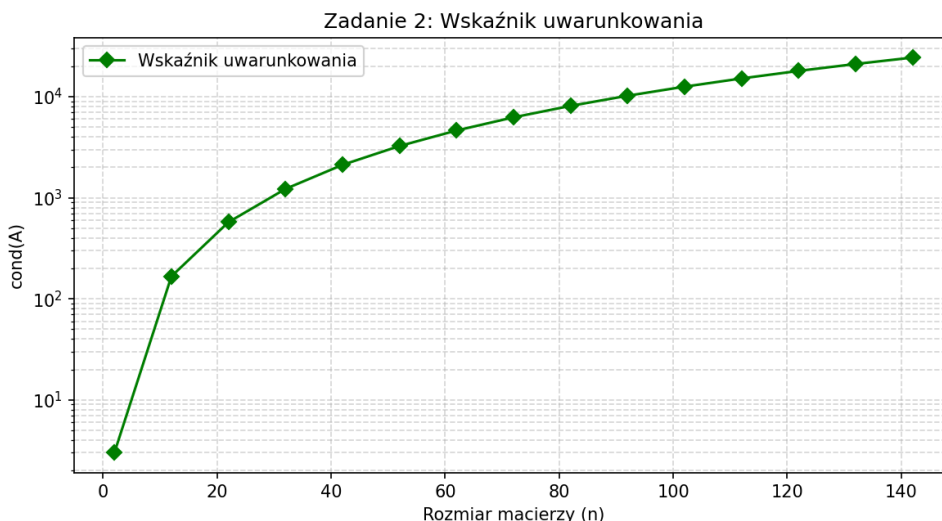
nalnym poziomie **zera maszynowego** ($\sim 10^{-13}$) nawet dla ogromnych rozmiarów układu $n = 150$.



Rysunek 3: Błąd wyznaczania niewiadomych

Wnioski:

- **Sukces eliminacji Gaussa:** Różnice wyliczonego wektora stabilizują się i pozostają w okolicach precyzji sprzętowej. Błąd zachowuje się płasko – niebieska krzywa oscyluje w poziomie **szumu numerycznego** (od 10^{-14} do 10^{-13}).
- **Pionowe kreski na początku:** Skrajne, pionowe tąpnięcia linii przy zerze to skutek **perfekcyjnego obliczenia wyniku** (brak żadnego błędu) dla $n = 2$, co w skali logarytmicznej powoduje matematyczny ubytek w nieskończoność ($\ln 0 = -\infty$) i odpowiada za taki efekt wizualny na wykresie.



Rysunek 4: Niskie uwarunkowanie macierzy

Wnioski merytoryczne z wykresów i danych:

- **Znacząca różnica uwarunkowania:** Zasadniczą różnicą między zadaniem 1 a 2 są rzędy wielkości na osiach Y (Rys. 4). Wskaźnik $\text{cond}(A)$ zachowuje tu niesamowicie **niskie wartości** (zaledwie 10^4 dla dużej, końcowej macierzy $n = 150$).

5. Analiza i wnioski: Złożoność algorytmów (Zadanie 3)

W ostatnim zadaniu badano układ z rzadką macierzą trójdziagonalną A_3 . Rozwiązano go klasyczną metodą eliminacji Gaussa oraz specjalnie dedykowanym do takich operacji – algorytmem Thomasa.

Zadanie 3 (Macierz pełna trójdziagonalna używana przez klasycznego Gaussa):

W poniższej tablicy `numpy.ndarray` bardzo ładnie widać, że poza trzema przekątnymi wszędzie indziej znajdują się zera. Stanowi to namacalne uzasadnienie, dlaczego dla tego typu układów musimy szukać lepszych algorytmów niż metoda Gaussa, która marnowałaby zasoby na sumowanie rzędów pustych zer.

$$A_3 = \begin{pmatrix} -9.0 & 1.0 & 0.0 & 0.0 & 0.0 \\ 2.0 & -13.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & 1.3333 & -17.0 & 3.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & -21.0 & 4.0 \\ 0.0 & 0.0 & 0.0 & 0.8 & -25.0 \end{pmatrix}$$

Zadanie 3 (Macierz pasmowa przechowywana na potrzeby algorytmu Thomasa):

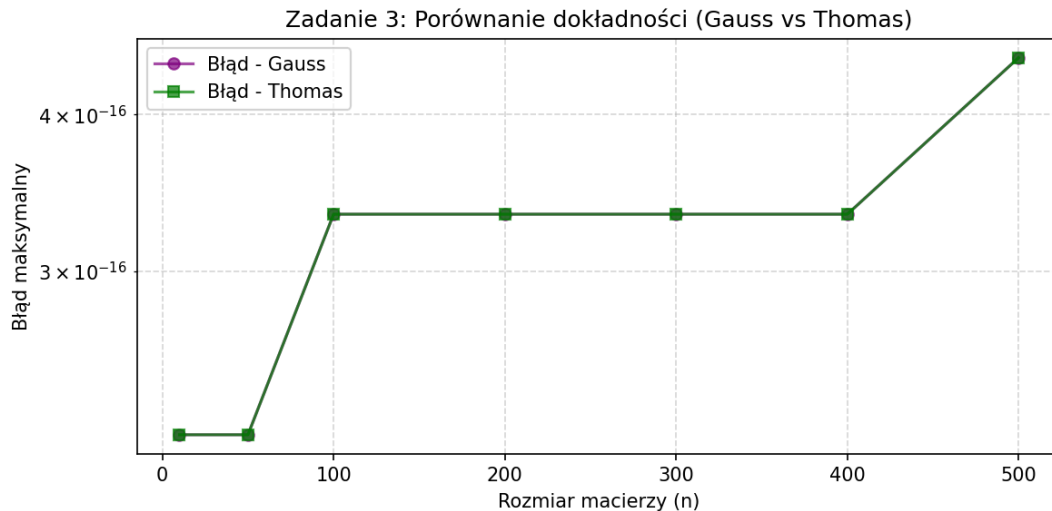
Dzięki przechowaniu w programie tylko 3 kolumn (odpowiednio: [poddziagonalna, diagonalna, naddziagonalna]), struktura zamiast gigantycznego, kwadratowego rozmiaru $n \times n$, ma zbitą i stałą wymiarowość $n \times 3$, co drastycznie oszczędza pamięć. Puste pola na skrajach zastąpiono strukturalnym zerem.

$$A_{\text{pasmowa}} = \begin{pmatrix} 0.0 & -9.0 & 1.0 \\ 2.0 & -13.0 & 2.0 \\ 1.3333 & -17.0 & 3.0 \\ 1.0 & -21.0 & 4.0 \\ 0.8 & -25.0 & 0.0 \end{pmatrix}$$

Tabela 3: Eliminacja Gaussa vs Algorytm Thomasa dla macierzy trójdziagonalnych

n	Błąd (Gauss)	Czas [s] (G)	Pam. [KB] (G)	Błąd (Thomas)	Czas [s] (T)	Pam. [KB] (T)
10	2.22e-16	9.88e-04	6.72	2.22e-16	1.34e-04	1.12
50	2.22e-16	1.86e-02	39.76	2.22e-16	6.08e-04	3.52
100	3.33e-16	7.29e-02	157.34	3.33e-16	1.09e-03	6.65
200	3.33e-16	2.95e-01	626.87	3.33e-16	2.16e-03	12.90
300	3.33e-16	6.52e-01	1408.90	3.33e-16	3.42e-03	19.15
400	3.33e-16	1.16e+00	2503.43	3.33e-16	4.77e-03	25.40
500	4.44e-16	1.81e+00	3910.46	4.44e-16	5.74e-03	31.65

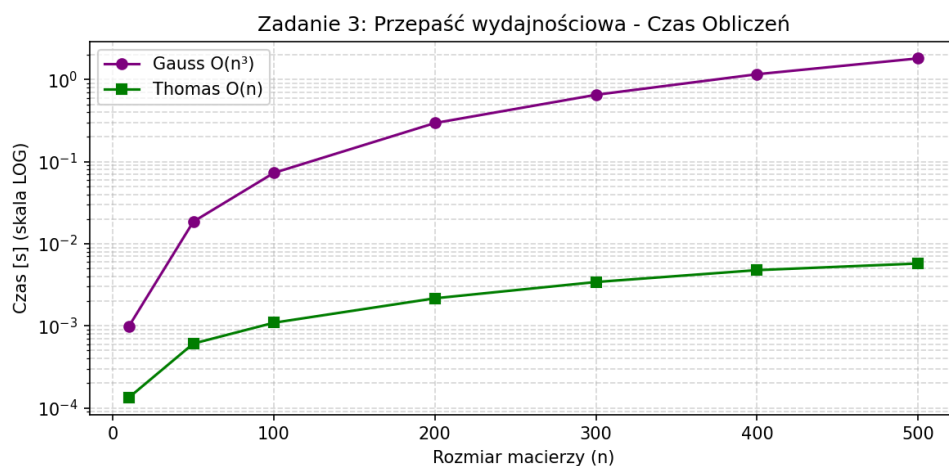
- **Dokładność:** Mimo ogromnych uproszczeń objętościowych, algorytm Thomasa i Gauss osiągały **całkowicie identyczny błąd** na poziomie szumu maszynowego.
- **Czas wykonania ($O(n)$ vs $O(n^3)$):** Algorytm Thomasa jest **drastycznie szybszy**. Dla układu wielkości $n = 500$ Thomas rozwiązuje całość w zaledwie **~ 5 milisekund**, podczas gdy metoda Gaussa trawi na to **niemal 2 sekundy**.
- **Zużycie RAM ($O(n)$ vs $O(n^2)$):** Eliminacja Gaussa wymaga alokacji tablic pełnych nieznaczących zer (na co proces Pythona zużywa dla $n = 500$ blisko **~ 3910 KB**). Thomas przechowuje tylko wektory o rozmiarze $n \times 3$, redukując pamięć do minimalnych **~ 31 KB**.



Rysunek 5: Identyczna precyzja obu algorytmów (błąd rzędu 10^{-16})

Wnioski:

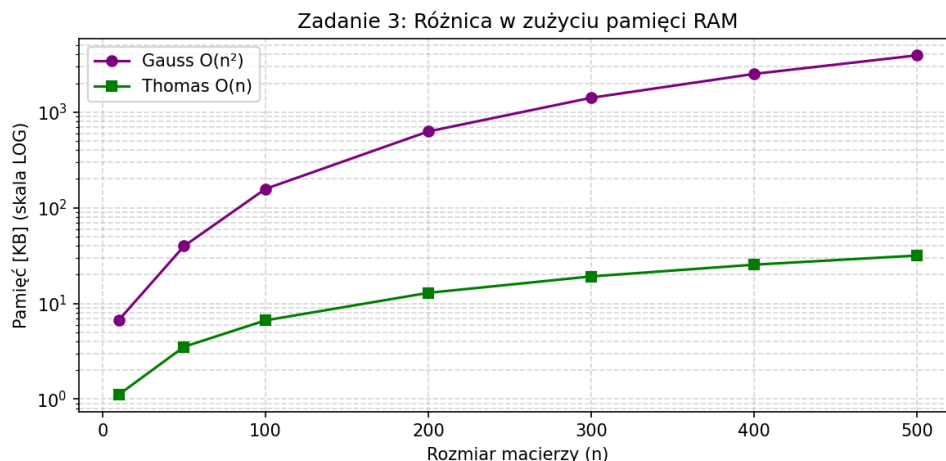
- **Tożsamość dokładności:** Wykres błęd (Rys. 5) pokazuje idealnie nałożone na siebie, płaskie linie w okolicach zera maszynowego (10^{-16}). To dowód na to, że unikanie zerowania liczb, które **już są zerami**, absolutnie nie degraduje precyzji obliczeń matematycznych.



Rysunek 6: Czas obliczeń (skala log)

Wnioski:

- **Przepaść wydajnościowa (Czas obliczeń):** Wykres czasowy (Rys. 6) obnaża wadę klasycznej eliminacji. Klasyczny Gauss wykazuje **złożoność sześcienną** $O(n^3)$. Algorytm Thomasa cechuje się **złożonością liniową** $O(n)$ – co w graficznej wersji dla rozmiarów na poziomie $n = 500$ tworzy tak gigantyczną przepaść, że do jej pokazania niezbędna była dziesiętna skala logarytmiczna.



Rysunek 7: Zużycie pamięci RAM (skala log)

Wnioski:

- **Implementacja w kodzie i profilowanie pamięci:** Widoczna na wykresie skrajna oszczędność pamięci wynika z konkretnego zastosowania programistycznego. Zamiast potężnej tablicy $n \times n$, do pętli Thomasa przekazano obiekt klasy `numpy.ndarray` o zablokowanym wymiarze $n \times 3$. Operacje odwołują się wyłącznie do kolumn 0, 1, 2 (nadpisując wartości w miejscu, ang. **in-place**). Profiler `tracemalloc` jasno wykreśla na osi, że dzięki takiemu układowi interpreter ładuje jedynie trzy ciągłe wektory, wymazując całkowicie asymptotyczny problem zużycia potężnej ilości RAMu ($O(n^2)$) w dużych układach macierzowych.

6. Podsumowanie końcowe i główne wnioski z laboratorium

Analiza wyników dla trzech zadań pozwala na wyciągnięcie uniwersalnych i bardzo istotnych praw numerycznych dotyczących algebry liniowej w środowisku maszynowym:

1. **Błąd numeryczny a złe uwarunkowanie problemu** Doświadczenie pierwsze dobitnie ukazuje granicę, przed którą staje programista w zderzeniu z matematyką. Brak prawidłowego wyniku przy metodzie Gaussa dla macierzy A_1 **nie jest błędem naszej implementacji ani ułomnością samego algorytmu**. Za potężne błędy odpowiada sama dziedzina zadania – macierz jest źle uwarunkowana z definicji. Z kolei **błąd numeryczny** to ułomność wynikająca ze skończonej liczby bitów w mantysie maszynowej. Laboratorium uczy, że użycie precyzyjniejszych typów (np. podbicie `float32` do `float64`) rozwiązuje błędy numeryczne, opóźnia nieznacznie objawy załamania algorytmu, ale w żaden sposób **nie leczy zepsutego uwarunkowania macierzy**. Wobec źle podstawionego zadania nawet idealny kalkulator będzie skazany na niepowodzenie.
2. **Dostosowanie algorytmu do specjalizacji macierzy (Thomas vs Gauss)** Trzecie zadanie stanowi fundamentalną lekcję z zakresu algorytmiki oraz wykorzystania struktury pamięci komputera. Jeżeli analizowany przypadek ukazuje powtarzalną specyfikację (jak trójdzielność, w której blisko 90% komórek macierzy nie niesie wartości analitycznej, lecz pełni funkcję sztywnych zer), klasyczne metody uniwersalne stają się **bezużyteczne wydajnościowo**. Implementacja Thomasa udowodniła, że dopasowanie matematycznej logiki pod odpowiednie ułożenie obiektów w pamięci (zmiana dwuwymiarowej tablicy na 3 równoległe wektory `numpy.ndarray`) przekształca algorytm z potężnego pożeracza zasobów $O(n^3)$, w szybki i bezbolesny dla pamięci RAM proces o koszcie zaledwie $O(n)$.